

Reproduzierbare Forschungsumgebungen

Ein GPU-fähiges Dev-Container-Template für die Arbeitsgruppe

Christian Heinzmann

Hochschule Heilbronn · Promotionskolleg

9. Juni 2026

Agenda

- 1. Motivation: Das Reproduzierbarkeitsproblem
- 2. Überblick: Was das Template liefert
- 3. Architektur: Drei Konfigurationsebenen
- 4. Reproduzierbarkeit & GPU
- 5. KI-Werkzeuge als Infrastruktur
- 6. Verteiltes Arbeiten: Laptop ↔ GPU-Server
- 7. Kritische Einordnung & Roadmap
- 8. Zusammenfassung

Teil 1

Motivation: Das Reproduzierbarkeitsproblem

Warum die Umgebung Teil der Wissenschaft ist

- **„Works on my machine“** — Ergebnisse hängen an undokumentierten lokalen Zuständen (Treiber, Bibliotheksversionen, Pfade).
- **Environment-Drift**: CUDA, Python und Paketversionen divergieren zwischen Personen und Maschinen — oft unbemerkt.
- **Onboarding-Kosten**: Tage, bis ein neues Mitglied überhaupt rechnen kann; jede Maschine ein Einzelfall.
- **Heterogene Hardware**: Laptop, Workstation, geteilte GPU-Server — dieselbe Pipeline, unterschiedliche Realität.

Kernaussage

Computational Reproducibility ist kein Nice-to-have, sondern Teil wissenschaftlicher Sorgfalt. Die **Umgebung muss versioniert, geteilt und identisch instanzierbar** sein — genau wie der Code selbst.

Teil 2

Überblick: Was das Template liefert

Drei Schritte, plattformunabhängig:

1. Repository klonen (inkl. Submodule)
2. In VS Code „*Reopen in Container*“
3. Loslegen — Umgebung ist fertig

Host-Voraussetzungen (das ist alles):

- Git · Docker (Compose V2)
- VS Code + Dev-Containers-Extension
- NVIDIA-GPU + Treiber (*optional*)

```
# 1) Klonen inkl. Submodul
git clone <repo-url>
git submodule update --init --recursive

# 2) VS Code: F1 ->
# "Dev Containers: Reopen in Container"

# 3) fertig - identisch auf
# Windows / macOS / Linux
```

Eine Spezifikation

Eine deklarative Definition erzeugt auf **jedem** Host dieselbe GPU-fähige, KI-unterstützte Arbeitsumgebung.

Was vorinstalliert ist

Stack

- Ubuntu 24.04 · CUDA 12.6.2 + cuDNN
- Python 3.12 in isoliertem venv ([/app/venv](#))
- Node.js 20 (für die Claude-Code-CLI)

KI-Werkzeuge ab Werk

- Claude Code (CLI + VS-Code-Extension)
- GitHub Copilot + Copilot Chat

Editor-Tooling

- ~30 VS-Code-Extensions automatisch
- Python/Pylance/Debugpy, vollständige Jupyter-Suite, Docker, Draw.io, Git-Graph
- Formatierung & Linting konfiguriert (100-Zeichen-Lineal als Coding-Rule)

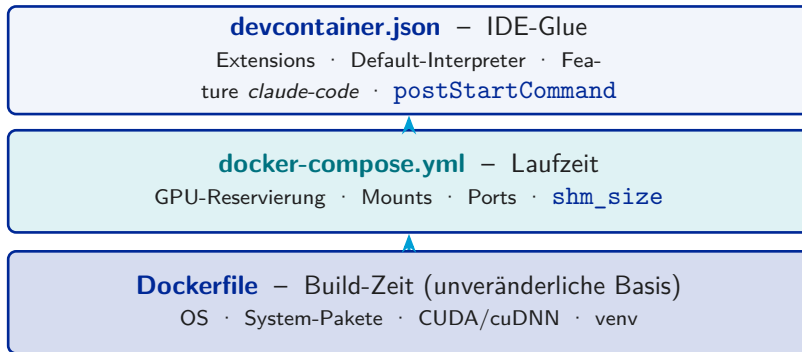
Ziel

Der erste Commit, nicht das Setup, ist der erste produktive Schritt.

Teil 3

Architektur: Drei Konfigurationsebenen

Trennung der Zuständigkeiten



Designprinzip: schwere System-Pakete werden ins Image *gebacken* (Build-Zeit), die Python-Abhängigkeiten bleiben zur *Laufzeit* die lebende Single Source of Truth.

Teil 4

Reproduzierbarkeit & GPU

Reproduzierbarkeit beginnt am OS-Layer

- Basis-Image **exakt gepinnt** — kein `latest`, sondern ein fixer Tag (rechts):
- Python systemseitig, aber in `venv /app/venv` isoliert — überall als Default-Interpreter gesetzt (Dockerfile, `devcontainer.json`, `settings.json`, Workspace).
- Node.js 20 *ausschließlich* für die Claude-Code-CLI.

```
FROM nvidia/cuda:12.6.2-cudnn-devel-ubuntu24.04

RUN apt-get update && apt-get install -y \
    python3.12 python3-pip python3.12-venv \
    git curl docker.io docker-compose-v2 \
    poppler-utils chromium ffmpeg ...

# venv als Default-Interpreter
RUN python3 -m venv /app/venv
ENV VIRTUAL_ENV=/app/venv
ENV PATH="/app/venv/bin:$PATH"
```

Konsequenz

Jede Person erhält denselben
CUDA / cuDNN / Python-Stack —
Reproduzierbarkeit startet am

- **Alle GPUs reserviert** über Compose `deploy.resources.reservations`.
- **shm_size: 32 GB** — löst den klassischen Shared-Memory-Engpass beim Daten-Loading.
- X11 durchgereicht (GUI-Apps), Port 4010 für Streamlit forwarded.
- Verifikation *vor* jeder Projektarbeit:

```
docker run --rm --gpus all \  
  nvidia/cuda:12.6.2-base-ubuntu24.04 nvidia-  
  smi
```

```
deploy:  
  resources:  
    reservations:  
      devices:  
        - driver: nvidia  
          count: all  
          capabilities: [gpu]  
shm_size: '32gb'  
volumes:  
  - ${localWorkspaceFolder:-.}/app/...  
  - ~/.gitconfig:/etc/gitconfig:ro  
  - ~/.ssh:/root/.ssh:ro
```

Automation: idempotenter Start

- `postStartCommand` ruft bei jedem Start `setup_startup.sh` (mit `set -e`).
- Schritte: venv aktivieren, `requirements` installieren, X11 freigeben, GitHub-SSH prüfen.
- **Non-fatal by design**: jeder optionale Schritt ist mit `|| true` abgesichert — ein fehlendes X-Display oder SSH-Key blockiert den Container nie.
- Credentials werden **wiederverwendet statt eingebrannt**: `~/.gitconfig` und `~/.ssh` read-only gemountet.

```
source /app/venv/bin/activate
python -m pip install --no-cache-dir -r .../requirements.txt
xhost +local:docker >/dev/null 2>&1 || true # GUI, best effort
ssh -o StrictHostKeyChecking=accept-new -T git@github.com || true # nur Info
```

Teil 5

KI-Werkzeuge als Infrastruktur

KI-Assistenten sind first-class Umgebung

- Claude Code wird **dreifach** bereitgestellt: devcontainer-Feature `anthropics/.../claude-code:1`, VS-Code-Extension und Node 20 im Image.
- **Ein Regelsatz für beide Assistenten:** `CLAUDE.md`, `.claude/settings.json` und `.github/copilot-instructions.md` verweisen auf *dieselbe* `CodingRules.md`.
- Single Source of Truth: Konventionen einmal pflegen — Claude Code *und* Copilot folgen automatisch.

Brücke

Die KI-Werkzeuge sind kein Add-on, sondern Teil der reproduzierbaren Umgebung.

Details in Präsentation 2 (Claude Project Template).

Teil 6

Verteiltes Arbeiten: Laptop $\leftrightarrow$ GPU-Server

Denselben Container überall ausführen

Variante A — alles remote

Workspace *und* Container auf dem GPU-Server (Remote-SSH + „Reopen in Container“).

Variante B — Code lokal, GPU remote

Docker-Context über SSH: der Laptop behält den Code, der Container läuft auf dem Server — **GPU ausleihen, ohne Code zu kopieren**.

```
docker context create gpu \  
  --docker host=ssh://gpu-server  
docker context use gpu
```

Begleitende Host-Playbooks

([docs/help/NewPC/](#))

- **Multi-Remote-Git**: ein `git push all main` → GitHub *und* Uni-GitLab gleichzeitig.
- **eduVPN Split-Tunneling**: nur das Campus-Subnetz durch den Tunnel (systemd-persistent).
- **SMB/CIFS-Auto-Mount**: `vers=3.0, nofail, x-systemd.automount`.

Kernaussage

Das Template adressiert die reale Topologie einer Arbeitsgruppe — nicht nur die ideale

Teil 7

Kritische Einordnung & Roadmap

Stärken

- Basis-Image am OS-Layer gepinnt
- GPU + ML-Defaults out-of-the-box
- KI-Tooling integriert & geteilt
- Remote-/Multi-Maschinen-fähig
- Umfangreich dokumentiert

Offene Punkte (Roadmap)

- **Python-Deps nicht gelockt** (nur `streamlit==1.42.2`) → Lockfile (`uv/pip-tools`, Hashes).
- **Keine CI** → Actions: Image-Build, `black/pylint`, `pytest`.
- **Doku-Drift**: ReadMe nennt CUDA 12.6.0/Ubuntu 22.04 & `python:3.9`; Submodul `src/Template` vs. Doku `src/NewWandB`.
- **Kein CPU-Fallback**: `count: all` hart kodiert; Doku verweist auf nicht existente `#NVIDIA`-Marker.
- **Sicherheit auf geteilten Servern**:

Konkrete nächste Schritte

1. **Lockfile einführen** — repräsentativen Dev-Stack (numpy, pandas, torch passend zu CUDA 12.6, pytest/black/pylint) vollständig pinnen, mit Hashes.
2. **Minimale CI** — Image bauen, Format/Lint/Tests laufen lassen, `main` mit Required-Checks schützen (macht die „getestet vor Merge“-Regel real statt aspirativ).
3. **Konsistenz-Pass** — ReadMes aus den echten Dateien speisen, Submodul-Namen vereinheitlichen, `.claude/worktrees/` aus der Versionierung nehmen.
4. **CPU-Profil** — Compose-Profile oder `docker-compose.cpu.yml`, damit auch reine Laptops starten.
5. **Härtungsprofil** dokumentieren — non-root-User mit Host-UID, SSH-Agent-Forwarding statt `~/.ssh`-Mount, `docker.sock` opt-in.
6. **FAIR-Anker** — `LICENSE`, `CITATION.cff`, Zenodo-DOI: vom Setup zum *zitierbaren* Artefakt.

Teil 8

Zusammenfassung

- Die **Umgebung wird zum versionierten Artefakt** — Reproduzierbarkeit ab dem OS-Layer.
- **Ein Klick** zur GPU-fähigen, KI-unterstützten Arbeitsumgebung, identisch auf jedem Host.
- **Skaliert** über Laptops und geteilte GPU-Server hinweg.
- Roadmap: *Lockfile + CI + Doku-Konsolidierung + DOI* — von „reproduzierbar in der Absicht“ zu „maschinell verifiziert“.

Fragen & Diskussion

Fortsetzung: Präsentation 2 — Claude Project Template